# Lab 11 — Commented answers

*Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.*

---

### Question 1 — Two Compose VMs versus a cluster

**Answer.** (1) **A VM goes down**: its containers are dead until a human intervenes; in a cluster, the *controller manager* notices that Pods are missing and the *scheduler* places them on a healthy node. (2) **Scaling up**: going from two to six instances means four VMs to prepare; in a cluster, `replicas: 6` and the scheduler spreads them. (3) **Updating**: `compose up` recreates the container, the service drops for a few seconds on each VM; the Deployment performs a *rolling update* (lab 12), new Pod ready before the old one is removed, driven by the controller manager. Add self-healing on **machine failure**, which `restart: always` does not cover.

**Why.** Compose acts on one machine, on demand; the control plane acts on all of them, continuously, from a desired state.

**Nuance.** For an internal application with no availability requirement, the two Compose VMs are a reasonable answer (lab 09). The difference is paid in complexity.

**Example.**

```
kubectl get deployment api -o jsonpath='{.spec.replicas} desired, {.status.readyReplicas}
ready'
```

---

### Question 2 — `apply`, `apply`, `create`

**Answer.** `pod/api created`, then `pod/api unchanged`, then `Error from server (AlreadyExists): … pods "api" already exists`. **Declarative**: the file describes a state, not an action; `apply` compares the desired state with the existing object and acts only on the difference. **Idempotent**: applying the same file N times yields the same result as once — which makes `apply` safe in CI, in a script, or after an outage. `create` is imperative: "create", so it fails if the object exists.

**Why.** `apply` records the last applied configuration (annotation `kubectl.kubernetes.io/last-applied-configuration`) and computes a three-way *patch*.

**Nuance.** Some fields of a Pod are **immutable** (the image, for instance): an `apply` that changes them fails; you must delete and recreate. Deployments (lab 12) exist precisely to change the image without that hassle.

**Example.**

```
kubectl apply -f pod-api.yaml ; kubectl apply -f pod-api.yaml      # created, then unchanged
kubectl get pod api -o jsonpath='{.metadata.annotations}' | cut -c1-80
```

---

### Question 3 — A lone Pod, a Pod under a Deployment

**Answer.** The last `get pods` shows **a single** Pod, `api-xxxxxxxxx-yyyyy`, created a few seconds ago; `web` is gone. `web` is a Pod **with no owner**: deleted, nobody asks for it back. The `api` Pod belongs to a **ReplicaSet**, itself created by the **Deployment**: the ReplicaSet's reconciliation loop sees `0 < 1` replica and creates a new one, with another suffix.

**Why.** Every object can carry an `ownerReference`; a controller watches the objects it owns and maintains the count.

**Nuance.** The new Pod has a **new IP** and a new name: nothing that pointed at the old one must have been hard-coded — the argument for the Services of lab 13.

**Example.**

```
kubectl get pod -l app=api -o jsonpath='{.items[0].metadata.ownerReferences[0].kind}'   # ReplicaSet
```

---

### Question 4 — `ImagePullBackOff` on a local image

**Answer.** The kubelet asked **its** containerd to pull `localhost/api:1.0`; that containerd does not know the image (it sits in Podman's storage, another directory, another user) and attempts a pull from a `localhost` registry that does not exist: `ErrImagePull`, then `ImagePullBackOff` (retries with growing delays). Fix: `podman save localhost/api:1.0 | sudo k3s ctr images import -`; the kubelet retries and finds the image. The field that matters: `imagePullPolicy: IfNotPresent` (or `Never`), otherwise `Always` would make even an imported image fail.

**Why.** Two runtimes, two stores: rootless Podman (`~/.local/share/containers`) and k3s's containerd (`/var/lib/rancher/k3s`).

**Nuance.** `Always` is the default when the tag is `latest` or absent; `IfNotPresent` for any other tag. Writing the policy explicitly avoids the surprise.

**Example.**

```
sudo k3s crictl images | grep api        # empty before the import, one line after
kubectl describe pod api | grep -A2 'Failed to pull'
```

---

### Question 5 — The invisible Pod

**Answer.** The Pod is in **another namespace** than the current context's (`default`). To see it: `kubectl get pods -n lab11`, or `-A` (all namespaces). To stop repeating `-n`: `kubectl config set-context --current --namespace=lab11`, which changes the context's default namespace in the kubeconfig.

**Why.** A namespace is a partition: every `kubectl` command applies to a single one, that of the `-n` option or of the context.

**Nuance.** Some objects are **not** namespaced (nodes, namespaces, StorageClasses, PersistentVolumes): `kubectl api-resources` says so in the `NAMESPACED` column.

**Example.**

```
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}'   # lab11
```

---

### Question 6 — One Pod or two

**Answer.** (a) nginx writes `proxy_pass http://localhost:8080/`: same network namespace, like the Podman pod of **lab 07**. (b) nginx writes `proxy_pass http://api:8080/`, a name resolved by a **Service**: like the user-defined network and Compose of **lab 09**. In production, (b): each service is

scaled, updated and restarted **independently**; with (a), three replicas of the front end would impose three replicas of the API, and updating one would restart the other.

**Why.** The Pod is the unit of placement and replication: what lives in the same Pod lives and dies together.

**Nuance.** Multi-container Pods serve *sidecars* (log agent, TLS proxy) and *init containers*: companions of the main container, not other services.

**Example.**

```
kubectl explain pod.spec.initContainers | head -5
```

---

### Question 7 — k3s's kubeconfig

**Answer.** Three elements: the **cluster** (`server: https://127.0.0.1:6443` and the authority's certificate), the **user** (a client certificate and its key, which is the cluster administrator), and the **context** `default` that ties them. In `644` mode, any user of the machine reads the key and becomes administrator: acceptable on your workstation where you are alone, unacceptable on a shared server. Copied elsewhere, the file points to `127.0.0.1`: you must replace the address with the k3s machine's (and port 6443 must be reachable) — the server certificate accepts the node's IP by default.

**Why.** The kubeconfig is both `kubectl`'s address book and its key ring.

**Nuance.** In the workplace, the kubeconfig holds a *token* or an OIDC authentication with a limited lifetime, not an eternal administrator certificate.

**Example.**

```
grep -E 'server:|client-certificate-data|name:' /etc/rancher/k3s/k3s.yaml | cut -c1-60
```

---

### Question 8 — `localhost:8080 refused`

**Answer.** `kubectl` finds **no kubeconfig** and falls back on its historical default, `http://localhost:8080`, where nothing listens (the API server is on HTTPS, port 6443). This happens when you use a separately installed `kubectl` (`apt`, `snap`), which does not know `/etc/rancher/k3s/k3s.yaml`. Fixes: `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (in `~/.bashrc`), or `mkdir -p ~/.kube && cp /etc/rancher/k3s/k3s.yaml ~/.kube/config`.

**Why.** The `kubectl` shipped by k3s is a link to the `k3s` binary, which knows where its kubeconfig is; a generic `kubectl` does not.

**Nuance.** Copying the file to `~/.kube/config` makes it **your** copy: if k3s regenerates its certificates, it becomes stale. The `KUBECONFIG` variable always follows the original.

**Example.**

```
which kubectl ; ls -l $(which kubectl)       # /usr/local/bin/kubectl -> k3s
KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get nodes
```

---

### Question 9 — API server down

**Answer.** The Pods **keep running**: the kubelet started them and watches them locally; traffic reaching them is served. You can no longer **change or observe** anything: `kubectl` fails, no new Pod gets scheduled, no Deployment gets reconciled. A Pod that dies is relaunched by the kubelet **on the same node** (it enforces the `restartPolicy`), but if another node is needed, nobody decides. The API server is therefore the door for **modifying and observing**; the real engine is the kubelet, which remembers what it must keep running.

**Why.** Kubernetes is designed so that the data plane survives the control plane: every node is autonomous for what it already has.

**Nuance.** With single-node k3s, "the API server stops" means `systemctl stop k3s`, which **also** stops the kubelet and containerd: the Pods die. Node autonomy only shows on a real cluster.

**Example.**

```
kubectl cluster-info          # Kubernetes control plane is running at https://127.0.0.1:6443
kubectl get --raw /healthz    # ok
```

---

### Question 10 — The Events of a start-up

**Answer.** `Scheduled` — the **scheduler** chose the node `wsl-ubuntu` (here the only one). `Pulling` / `Pulled` — the **kubelet** asked containerd to pull the image (3.75 s). `Created` / `Started` — the kubelet created then started the container through containerd. Against `podman run` from lab 03: the pull, the creation and the start are the same steps, done by `podman` and `crun`; the cluster adds **scheduling** (choosing a machine), **recording** each step as an Event in the API server, and **continuous supervision**. With a single node, the scheduler has no choice: it "works" without deciding.

**Why.** Each component publishes Events on the object it handles; `describe` gathers them in order.

**Nuance.** Events are kept for one hour by default: for an older incident, you need the application's logs or an external collection.

**Example.**

```
kubectl get events --sort-by=.lastTimestamp | tail -5
```

---

### Question 11 — `Pending` with no IP

**Answer.** Either **no node fits** (insufficient resources, node `NotReady`, placement constraint), or **the image or a volume blocks** before assignment — but with no IP at all, it is almost always the scheduler. `kubectl describe pod` settles it: `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory` on one side; `Scheduled` followed by a `Failed to pull image` or a `FailedMount` on the other (with an IP already assigned).

**Why.** The IP is assigned by the CNI when the kubelet creates the Pod on a node; no node, no IP.

**Nuance.** A Pod requesting `resources.requests.memory: 8Gi` on a 4 GB WSL will stay `Pending` forever, with no error message anywhere but in its Events.

**Example.**

```
kubectl describe pod api | grep -A3 '^Events' ; kubectl get nodes ; kubectl describe node |
grep -A5 'Allocated resources'
```

---

### Question 12 — From k3s to EKS

**Answer.** To request or configure: (1) an **Ingress controller** (Traefik ships with k3s; on EKS you install ingress-nginx or use the ALB); (2) a default **StorageClass** (`local-path` with k3s; on EKS, EBS through a CSI driver); (3) `LoadBalancer` Services (ServiceLB with k3s; a paid cloud load balancer on EKS); (4) **image access**: no more `k3s ctr images import`, images come from a registry (ECR) with pull rights. What does not change: the **manifests** (Pod, Deployment, Service, ConfigMap…) and the `kubectl` commands.

**Why.** k3s bundles choices that managed clusters leave open, because they depend on the customer's infrastructure.

**Nuance.** Even on EKS, the platform team has usually already made those choices: the developer **discovers** them (`kubectl get storageclass`, `kubectl get ingressclass`) rather than installing them.

**Example.**

```
kubectl get storageclass ; kubectl get ingressclass ; kubectl get svc -n kube-system traefik
```

---

### Question 13 — `hostPath` and `hostPort`

**Answer.** `hostPath` mounts a file **from the machine running the Pod**: on a cluster, the Pod can land on any node, where that file does not exist — and a Pod that reads the nodes' disk is a security hole (a `hostPath` on `/etc` reads the host's configuration). `hostPort` reserves a port **of the node**: two replicas on the same node clash, and the client must know the node's address. Replacements: the nginx configuration becomes a **ConfigMap** mounted as a volume (lab 14); exposure becomes a **Service** then an **Ingress** (lab 13), which depend on no node.

**Why.** A cluster manifest must be **machine-independent**: everything the Pod needs is an API object, not a local path.

**Nuance.** `hostPath` and `hostPort` have legitimate uses for **node agents** (log collector, monitoring) deployed as a DaemonSet, one per machine.

**Example.**

```
grep -n -E 'hostPath|hostPort' ~/labo-docker/11/demo-pod.yaml
```